Online gaming systems require efficient sorting algorithms to manage player rankings, leaderboards, and match-mal
sorting algorithms, performance analysis, online gaming, factorial experiment, merge sort, quick sort, data distribut

Introduction

Online gaming systems have experienced unprecedented growth, with millions of concurrent players generating mas
The choice of sorting algorithm significantly impacts system performance, particularly when dealing with gaming-sp
This paper addresses the critical need for empirical performance analysis of sorting algorithms in gaming contexts.

Research Objectives

Our study aims to:

Evaluate the performance of Merge Sort and Quick Sort on gaming-relevant data distributions
Analyze the impact of data size and distribution patterns on algorithm performance
Provide empirical guidelines for algorithm selection in online gaming systems
Quantify the trade-offs between execution time, memory usage, and throughput

Contributions

The main contributions of this work include:

A comprehensive factorial experiment design (2^3) for sorting algorithm evaluation
Implementation of realistic gaming data generators using inverse transform sampling
Detailed statistical analysis including ANOVA, confidence intervals, and effect size calculations
Practical recommendations for algorithm deployment in production gaming systems

Related Work

Sorting algorithms have been extensively studied in computer science literature [0]. However, most studies focus on
Recent work by [0] examined database query optimization for gaming systems but did not address in-memory sortin
Our work differs by focusing specifically on the intersection of sorting algorithm performance and gaming data char

Methodology

Experimental Design

We employed a full factorial experiment design (2^3) to systematically evaluate the performance of sorting algorithm

Factor A - Data Size: 10,000 vs 100,000 elements
Factor B - Data Distribution: Exponential vs Quasi-sorted
Factor C - Algorithm: Merge Sort vs Quick Sort

This design resulted in 8 unique combinations, with each experiment repeated 10 times to ensure statistical reliabili

Data Generation

Exponential Distribution Gaming scores typically follow exponential distributions, where most players achieve mod

where $U \sim Uniform(0, 1)$ and $\lambda = 0.001$ to achieve a mean score of approximately 1,000 points.

Quasi-sorted Distribution Leaderboards in gaming systems are frequently updated, creating datasets that are nearly

Algorithm Implementation

Merge Sort We implemented a standard divide-and-conquer Merge Sort with $O(n \log n)$ guaranteed time complexity
Quick Sort Our Quick Sort implementation uses the median-of-three pivot selection strategy to improve performanc

Performance Metrics

For each experiment, we measured:

Execution Time: Wall-clock time using high-precision timers
Memory Usage: Peak memory consumption during sorting
Throughput: Elements processed per second
Speedup: Relative performance between algorithms

Experimental Environment

The experiments were conducted on:

Operating System: Windows 11
Processor: AMD Ryzen 7 5800H with Radeon Graphics (8 cores, 16 threads)
Memory: 16GB DDR4
Python 3.12.7 with optimized libraries
Each execution in isolated process to prevent interference
Controlled environment with dedicated resources to ensure reproducibility

Statistical Analysis

We conducted comprehensive statistical analysis including:

Descriptive statistics with confidence intervals (95%)
Analysis of Variance (ANOVA) to identify significant factors
Post-hoc analysis with Bonferroni correction
Effect size calculation using Cohen's d
Outlier detection using the IQR method

Results and Analysis

Distribution Analysis of Execution Times

Figure presents a detailed analysis of execution time distributions through box plots, showing performance differen
[H] [width=0.45]plots/boxplots$_d istribuicao.pngBoxplotanalysisshowingdistributionofexecutiontimesbyalgorithm$,

Overall Performance Comparison

Figure presents a direct comparison between Merge Sort and Quick Sort across different experimental configuration
[H] [width=0.45]plots/comparativo$_b arras.pngPerformancecomparisonbetweenMergeSortandQuickSortacrossdif$

Scalability Analysis

Figure demonstrates how each algorithm scales with increasing data size.
[H] [width=0.45]plots/escalabilidade$_a lgoritmos.pngScalabilityanalysisshowingalgorithmperformanceasdatasizei$n

Performance Metrics Correlation

Figure presents the correlation matrix between different performance metrics.
[H] [width=0.42]plots/heatmap$_c orrelacao.pngCorrelationheatmapbetweenperformancemetrics(time, memory, thr$

Factor Interaction Analysis

Figure shows how different experimental factors interact with each other.
[H] [width=0.45]plots/interacao$_f atores.pngInteractionanalysisbetweenexperimentalfactors(algorithm, size, distr$

Memory vs Time Trade-off